
boolean

Dimensioni

Width e Height

Permettono di impostare la larghezza e l'altezza del contenuto degli elementi.

Ci sono numerose unità di misura utilizzabili, ma le unità più comunemente usate sono:

- **px** (pixel)
- **%** (percentuale rispetto al blocco contenitore)
- **em** (basato sulle dimensioni del font)

```css
div {
  width: 250px;
  height: 800px;
}
```

Abbiamo anche la possibilità di definire un minimo e un massimo di altezza e larghezza, utilizzando:

- **`max-width`**(larghezza minima)
- **`min-width`** (larghezza minima)
- **`max-height`** (altezza massima)
- **`min-height`** (altezza minima)

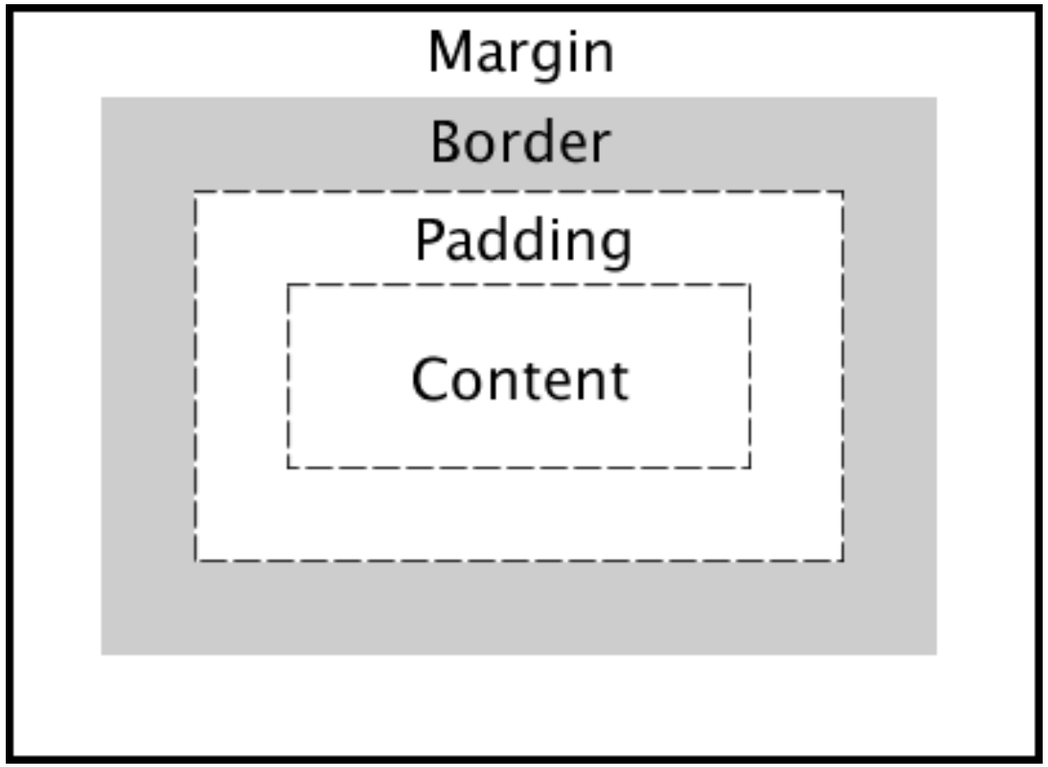


Box Model

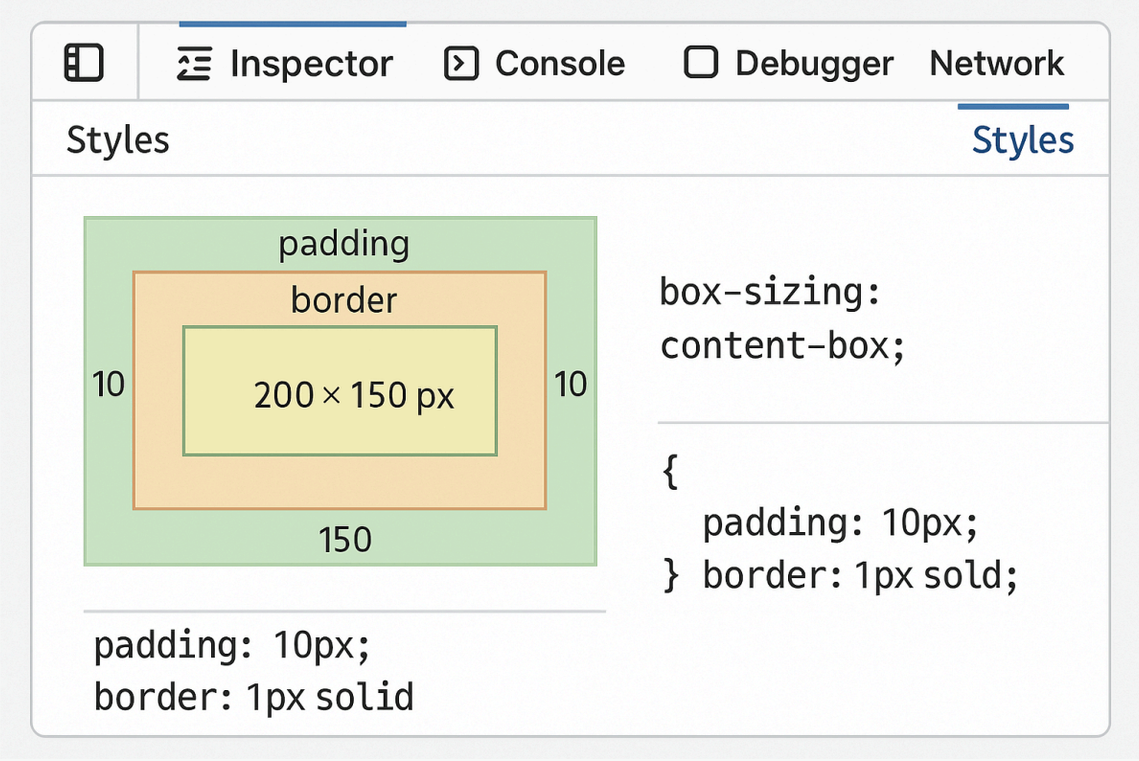
Come il CSS vede il mondo

Bordi, Margini e Padding, cosa sono?

Surprise surprise, non è così banale



BROWSER INSPECTOR



Il nostro più valido alleato per controllare tutto ciò che succede nel browser.

Ricordate: questa è l'unica fonte di verità!

Border

Tutti i lati assieme

Border imposta lo spessore, stile e il colore del bordo del blocco.

Lo spessore viene indicato con un'unità di misura, preferibilmente in pixel, oppure con le parole chiave thin, medium e thick.

Lo spessore del bordo si somma alle dimensioni totali del blocco per stabilire lo spazio occupato nella pagina.

Il colore viene indicato con le stesse notazioni viste per background.

```css
div {
  border-width: 1px;

  border-style: solid;

  border-color: black;

  /*Shorthand */
  border: 1px solid black;
}
```

Border

Un lato alla volta

```css
div {
  border-top: 1px solid black;

  border-right: 2px solid black;

  border-bottom: 1px solid black;

  border-left: 3px solid black;
}
```

Border

Gestiamo le proprietà dei bordi

Se si specificano due valori:
il primo valore si applica ai lati superiore e inferiore, il secondo valore si applica ai lati destro e sinistro.

Se si specificano tre valori:
il primo valore si applica al lato superiore, il secondo ai lati destro e sinistro, il terzo valore al lato inferiore.

Se si specificano quattro valori:
i valori si applicano nell'ordine al lato superiore, destro, inferiore e sinistro.

```
1 div {
2   border-style: solid;
3
4   border-width: 2px 3px;
5
6   border-color: blue red yellow pink;
7 }
```

Border Radius

Se gli spigoli non ci piacciono

Si possono impostare angoli arrotondati per i bordi del blocco, contemporaneamente o singolarmente.

```
1  div {
2    border-top-left-radius: 2px;
3
4    border-top-right-radius: 2px;
5
6    border-bottom-right-radius: 2px;
7
8    border-bottom-left-radius: 2px;
9
10
11   /*Shorthand*/
12   border-radius: 2px;
13 }
```

Padding

Lo spazio tra il bordo e il contenuto

Padding

Il **padding** imposta la distanza tra il contenuto ed il bordo del blocco.

Può essere indicato in uno dei seguenti modi:

- Con un'unità di misura, preferibilmente in pixel
- Come percentuale delle dimensioni del "contenitore"

Valgono le stesse regole già dette per **border** per quanto riguarda impostare singolarmente ogni lato e sulle forme abbreviate.

```css
div {
  padding: 10px;

  padding: 2%;
}
```



Di default le dimensioni del padding si sommano alle dimensioni di width e height del blocco, quindi aumentano le dimensioni totali del blocco.



Margin

Lo spazio tra elementi diversi

Margin

Specifica la distanza tra il bordo di un elemento ed il bordo degli elementi adiacenti.
Il suo valore può essere indicato in uno dei seguenti modi:

**Sono consentiti valori negativi ma solo in casi estremi!*

- Con un'unità di misura, preferibilmente in pixel
- Come percentuale delle dimensioni del "contenitore"
- Con la parola chiave auto, che indica al browser di calcolare automaticamente la distanza

Inoltre, valgono le stesse regole già dette per **border** per quanto riguarda impostare singolarmente ogni lato e sulle forme abbreviate.

```css
div {
  margin: 10px;

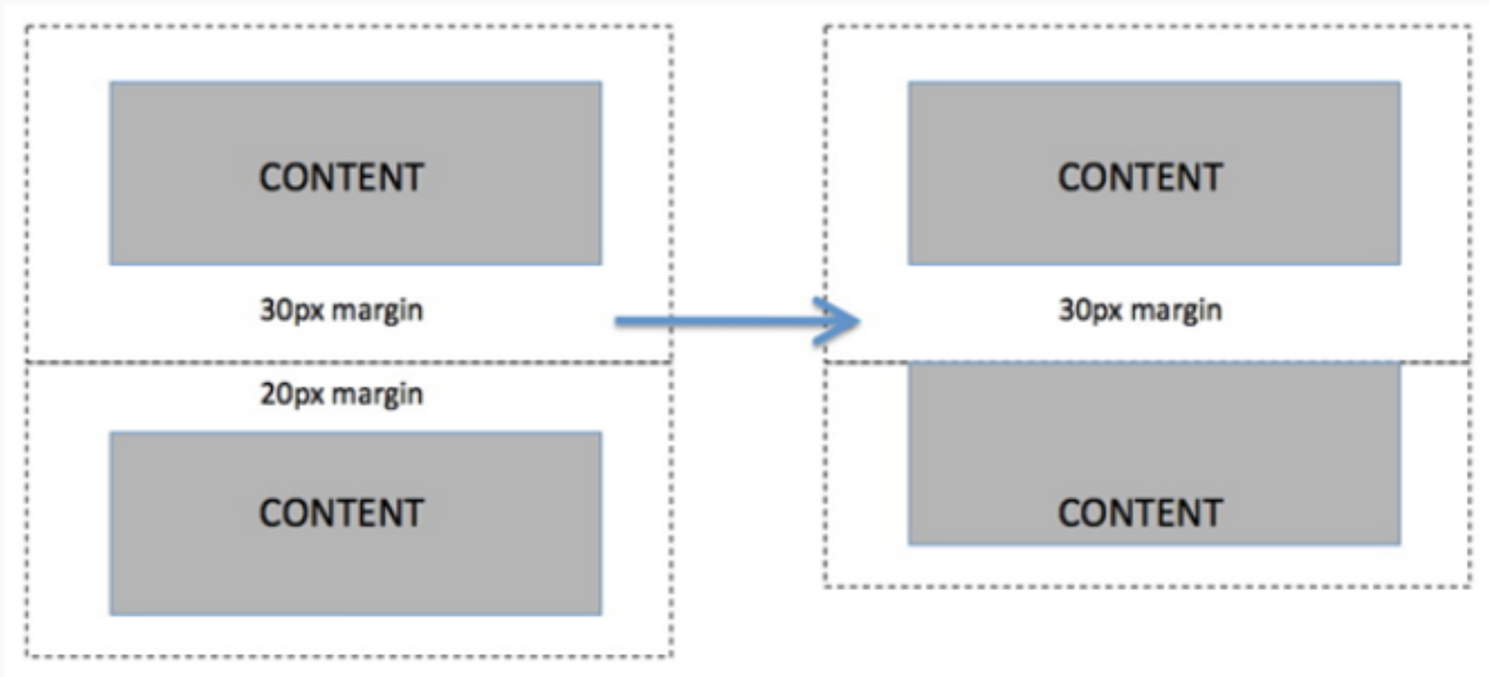
  margin: 2%;

  margin: 0 auto;
  /* "auto" funziona solo in orizzontale */
}
```

Collasso dei Margini

AKA "come perdere ore di lavoro e l’accesso al paradiso"

Quando due blocchi all'interno di una pagina si trovano uno direttamente sopra l'altro, sovente, la distanza tra i blocchi non è pari alla somma dei loro margini.
La distanza è, invece, pari al margine di valore maggiore tra i due.





Box Sizing

Come il CSS calcola le misure di un elemento

Box Sizing

La rivoluzione nel calcolo del dimensionamento dei blocchi.

Prima dell'introduzione del concetto di box-sizing, il dimensionamento dei blocchi avveniva solo in una modalità:
width ed height determinano la dimensione del contenuto, e padding e border-width vengono ad esso aggiunti per stabilire lo spazio realmente occupato dal blocco.

Adesso è possibile utilizzare due metodi diversi di **box-sizing**:

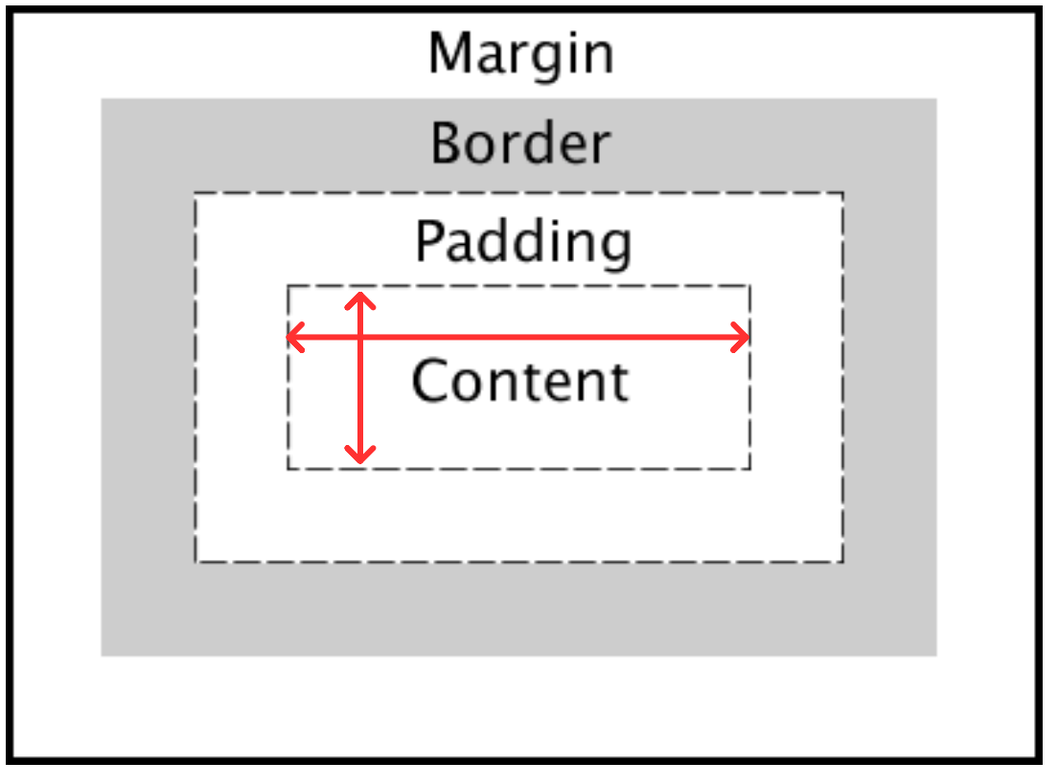
- **content-box**
  la "vecchia" modalità, di default.
- **border-box**
  padding e border vengono sottratti/assorbiti da width e height.
  In pratica un blocco con width: 200px occuperà sempre e solo 200 pixel nella pagina.

Box sizing

Cosa comprendono width ed height?

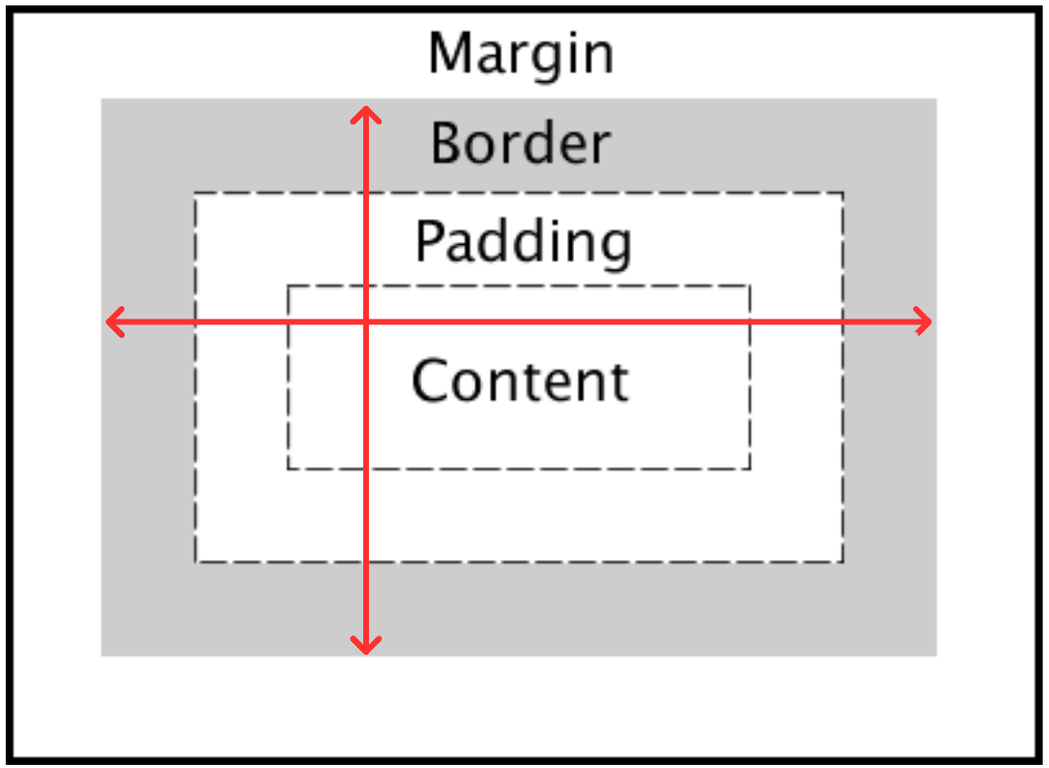
box-sizing: content-box

Width e height si riferiscono solo al contenuto



box-sizing: border-box

Width e height si intendono "bordo e padding compresi"



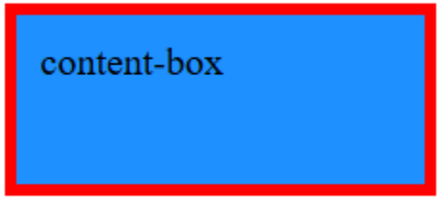
Box sizing

Cosa succede a parità di misure?

https://codepen.io/boolean-code/pen/QwNgGrE?editors=1100

```css
.box-1 {
  width: 150px;
  height: 50px;
  padding: 10px;
  border: 5px solid;
  box-sizing: content-box;
}
```

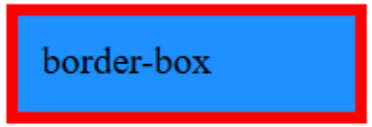
Altezza totale: **80px**
Larghezza totale: **180px**
Contenuto: **150px** x **50px**



I 30 pixel dati dalla somma di padding e bordi, **vengono aggiunti** alle dimensioni del contenuto

```css
box-2 {
  width: 150px;
  height: 50px;
  padding: 10px;
  border: 5px solid;
  box-sizing: border-box;
}
```

Altezza totale: **50px**
Larghezza totale: **150px**
Contenuto: **120px** x **20px**



I 30 pixel dati dalla somma di padding e bordi, **vengono sottratti** dalle dimensioni del contenuto

CSS Reset

La potenza è niente senza controllo!

Per mantenere il controllo **totale** sullo stile, si può ricorrere a dei fogli detti di "reset", che annullano qualunque stile applicato di default dai browser. (Esempio)

Molto spesso, ci basta mantenere il controllo sulle spaziature e sul box model e pertanto si ricorre alla soluzione che vedete a lato, che possiamo inserire in cima al nostro foglio css.

```css
/*
Reset per:
- Azzerare tutti i margini
- Azzerare tutti i padding
- Impostare a tutti i tag il box-sizing di tipo border-box
*/

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

/* Resto del CSS... */
```



Display

Come un elemento deve comportarsi nel flusso della pagina

Diversi tipi di tag

Block level tags

Crea un elemento di tipo "block" che occupa una intera linea; gli elementi successivi occuperanno una nuova linea (es. <p>).

Inline tags

Gli elementi occupano solamente lo spazio del loro contenuto; Altri elementi inline possono affiancarsi sulla stessa linea.

Self closing tags

Tag che possono essere sia Inline che Block ma che non prevedono contenuto (un testo, un titolo); per questo motivo non hanno apertura e chiusura. Ad esempio
 che non ha contenuto ma obbliga il testo ad andare a capo.

```
1 <p>Block tag</p>
2
3 <strong>Inline tag</strong>
4
5 <br> <!-- Self closing tag -->
```

Display

Travestiamo i nostri elementi...

La proprietà display permette di specificare quale modalità va usata per visualizzare l'elemento.
I valori più comunemente utilizzati per display sono:

- **none**: l'elemento non viene visualizzato e non occupa alcuno spazio all'interno del documento. Dal punto di vista del layout praticamente non esiste.
- **block**: ha comportamento di blocco in quanto a flussso e ingombro, e rende possibile l'impostazione di proprietà tipiche degli elementi blocco: width, height, etc..
- **inline**: l'elemento non interrompe il flusso del testo, non crea una nuova riga, viene affiancato ad altri elementi inline e non può contenere al suo interno degli elementi block.
- **inline-block**: rende possibile l'impostazione di proprietà tipiche degli elementi block, ma non interrompe il flusso del testo, non crea una nuova riga nel punto in cui viene inserito e viene affiancato ad altri elementi inline o inline-block.

Display

Display che vai, comportamento che trovi

Come imparerete a capire, spesso una proprietà CSS influenza il comportamento di altre regole.

Ad esempio, un elemento **inline** ignora le istruzioni di **margin-top** e **margin-bottom**, nonché di **width** e **height.**

Provare per credere: incolliamo il codice qui a fianco.

```css
*{
  margin:0;
  padding: 0;
  box-sizing: border-box;
}

p {
  background-color: red;
  display: inline;
  width: 100px;
  height: 100px;
  margin: 10px;
}
```

Approcci alla progettazione

Due approcci alla progettazione dello stile

Desktop First vs Mobile First

Desktop First
È l'approccio tradizionale: si progetta prima il sito web per gli schermi grandi (computer desktop), poi si adatta per dispositivi più piccoli come tablet e smartphone. È come partire da una casa grande e poi cercare di farla entrare in spazi sempre più piccoli.

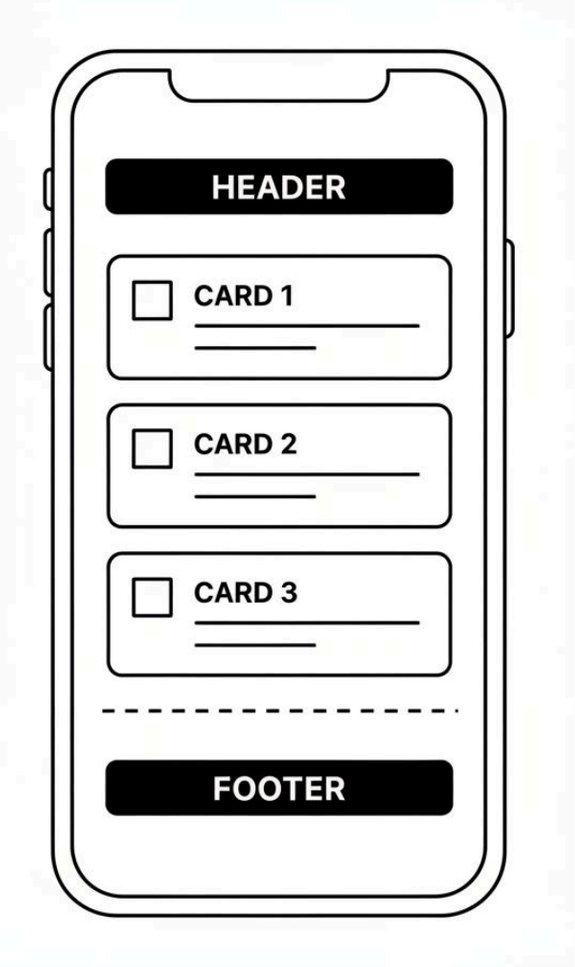
Mobile First
È l'approccio moderno: si progetta prima per gli smartphone (schermo piccolo), poi si espande per tablet e desktop. È come partire da un appartamento e poi aggiungervi stanze man mano che si ha più spazio. Questo metodo funziona meglio perché oggi la maggior parte delle persone naviga da mobile.

Due approcci alla progettazione dello stile

Partiamo dal Mobile First

Noi partiremo dal mobile first, che ci permetterà di concentrarci meglio sui singoli "componenti" delle nostre pagine e di preoccuparci di incolonnamenti e layout, solo quando saremo già bravi a curare lo stile dei singoli elementi delle nostre pagine.

Questo approccio, infatti, segue il flusso naturale degli elementi che si dispongono per default uno sotto l'altro!

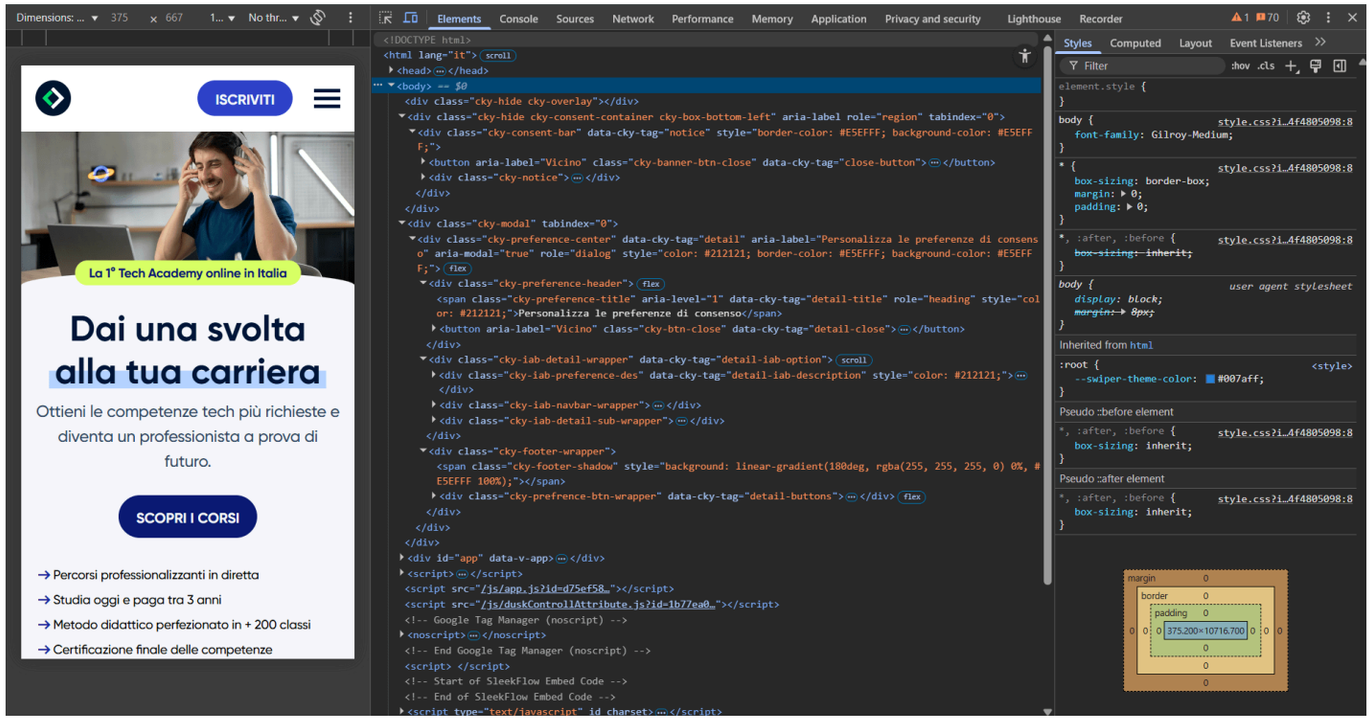


Mobile First

Inspector to the rescue!

Utilizzando il browser inspector, possiamo simulare lo spazio a disposizione, e visualizzare il tutto come se sviluppassimo su uno smartphone.

Questo ci consente inoltre di tenere sempre aperto l'inspector per controllare il nostro codice e fare delle prove in tempo reale!





LIVE CODING

ESERCITAZIONE